同济大学

毕业设计 (论文) 开题报告

课题名称	面向代码智能体的开发任务上下文追踪与代码变更溯源系统设计与实现
副 标 题	
学　　院	计算机科学与技术学院
专　　业	计算机科学与技术
学生姓名	待填写
学　　号	待填写

_____ 年 ____ 月 ____ 日

摘要

开发者借助代码智能体完成修改、执行与测试已成日常，但对话、变更、工具调用与测试结果分散在不同界面，缺少以开发任务为中心的组织，使“为什么这样改、依据什么验证、中断后如何接上”难以回答。本课题面向单个开发者，研究任务、对话、变更、工具调用与测试的关联与组织方法，实现支持任务上下文追踪与变更溯源的 Web 原型并评估其作用。研究定位为小样本探索性研究，结论以 6—8 个任务与 4—5 名受试者的描述统计表述。

关键词：代码智能体；任务上下文；变更溯源；中断恢复；证据链

一、毕业设计（论文）课题背景

1．课题来源及研究的目的和意义

课题基本信息见表 1-1。

表 1-1 课题基本信息

项	内容
课题中文名称	面向代码智能体的开发任务上下文追踪与代码变更溯源系统设计与实现
课题英文名称	Design and Implementation of a Task-Context Tracking and Code-Change Provenance System for Code Agents
课题性质	本科毕业设计（工程设计，含研究性内容）
课题来源	自拟
应用场景	单个开发者使用代码智能体完成日常软件开发任务
起止时间	2026 年 11 月—2027 年 5 月
学生 / 学号 / 班级	待填写
指导教师 / 职称	待填写

SWE-bench 以真实 GitHub issue 检验端到端修复，其论文报告当时最优模型仅解决 1.96% 的任务 [1]；针对代码生成能力的系统评测给出了更一般的基准结果 [2]；此后 SWE-agent[3]、OpenHands[4]、CodeAct[5] 等以工具调用与执行反馈为核心的范式显著提升了该指标，MetaGPT[6] 与 ChatDev[7] 探索多智能体协作，也有工作主张以更简单流程取得可比效果 [8]；综述从软件工程全流程 [9-10]、检索增强 [11] 与测试生成 [12] 等角度总结了进展。使用代价并未随能力提升而消

失：有对照实验报告 Copilot 使任务完成速度提升约 55.8%[13]，但开发者仍需额外审查与验证 [14-15]，采纳生成代码前普遍存在理解与验证环节 [16-17]。可见智能体的产出必须伴随可核查的过程证据。

现实困难在于过程信息分散：一次多轮修改涉及会话记录、代码差异、终端输出与测试报告四类界面，彼此没有共同标识；任务跨会话时历史按时间平铺，缺少任务级聚合；从代码反查“由哪句话触发、由哪次测试验证”只能人工回溯；中断后重启需重读大量上下文。

本课题的意义：方法上把“任务上下文”迁移到智能体驱动的新场景并给出与 PROV 的映射；工程上把关联、溯源与恢复做成可查询、可复现的原型；教学上完整走通“问题识别—文献定位—模型设计—原型实现—实证评估”的流程，并对小样本评估如实取舍。

2．国内外在该方向的研究现状和发展趋势

（一）变更分析与代码历史

软件仓库挖掘给出了变更与问题的关联方法：SZZ 以修复提交所引缺陷报告回溯引入缺陷的变更 [18]，Kagdi 等综述了仓库挖掘的任务与方法 [19]，Robbes 与 Lanza 主张以变更为中心分析演化 [20]；GumTree 以语法树差分给出比行级 diff 更精确的变更脚本 [21]，CodeShovel 重建方法级代码历史 [22]。这些方法回答“改了什么”，关联对象是提交与缺陷报告，而非“由哪句指令触发、由哪次测试验证”。

（二）任务上下文与中断恢复

以任务为单位组织开发上下文并非新命题。Mylyn 提出任务上下文模型，以任务聚焦界面按当前任务过滤 IDE 代码元素 [23]；TaskTracer 在桌面层把文件、邮件与网页归集到任务 [24]；ActivitySpace 跨应用推荐相关代码元素 [25]；Codebook 建立代码实体与提交、缺陷、邮件的关联图 [26]。恢复侧，Parnin 与 Rugaber 归纳恢复策略对线索丰富度的依赖 [27]，Parnin 与 DeLine 用受控实验比较情景与语义线索 [28]，Safer 与 Murphy 研究任务边界识别界面 [29]，Robbes 与 Lanza 刻画开发会话结构 [30]。这些工作是本课题基础，但其前提是开发者直接在 IDE 或桌面操作、任务由人显式声明，线索来自编辑与窗口事件。

（三）过程溯源与证据链

数据库领域给出溯源的经典框架 [31]；PROV-O 把溯源抽象为实体、活动与代理及其派生关系，成为跨系统交换标准 [32]；PROV-AGENT 覆盖智能体交互中的提示、响应、决策与工具调用 [33]。Whyline 以“为什么”提问驱动执行证据呈现 [34]，noWorkflow 把脚本执行捕获为可查询溯源图 [35]。这些工作证明证据链的价值，但未面向“指令—工具调用—变更—测试”这一异构序列建模，也未把验证依据纳入证据图。

（四）研究述评

工程侧工具亦可归入上述方向：智能体客户端（Claude Code[36]、Cursor[37]、Cline[38]）以会话或文件为单位提供历史与检查点，检查点随会话结束或保留期到期失效；LLM 可观测性平台（Langfuse[39]）与 OpenTelemetry 生成式 AI 语义约定 [40] 记录调用链与耗时，不与代码变更、测试结果语义对齐。上述工作可归为四个方向：智能体能力、变更分析、任务上下文与中断恢复、过程溯源。四者各有积累，但都未同时满足三点：（1）以任务为根统一挂接对话、变更、工具调用与测试；（2）变更粒度到 hunk 且支持双向查询；（3）在证据绑定约束下用大模型生成摘要与进展整理。需说明的是，“以任务为单位组织上下文”本身并非空白，Mylyn[23] 与 Parnin 等 [27-28] 已给出模型与实证基础；本课题差异在于对象从“人在 IDE 中的编辑事件”变为“智能体驱动的指令—工具调用—变更—测试序列”，并把验证依据纳入证据链。在所及文献范围内尚未见到同时满足三点的系统；文献综述定稿阶段将补做系统检索并记录检索式与命中数。

二、毕业设计（论文）方案介绍

1．主要研究内容

（一）研究问题

本课题围绕“单个开发者使用代码智能体时，开发过程信息如何被组织、溯源与恢复”提出五个可检验的研究问题。

RQ1（表示）：统一事件模型能否在不修改采集端的前提下覆盖一种真实代码智能体来源（Claude Code）与 Git/测试记录的全部类型？以事件类型覆盖率与

文件级位置映射成功率度量。

RQ2（关联）：融合时间邻近、显式引用与文件级位置映射的关联方法，在20—30 对人工核对样本上的关联准确率是多少？

RQ3（溯源）：以 hunk 为粒度、绑定触发消息与验证测试的证据链，能否在15 条预注册查询上达到可用命中率？大模型摘要的弃答率与不支持率如何？

RQ4（续接）：由进展摘要、未完成项与待验证修改构成的最小上下文，能否缩短 4—5 名受试者的恢复用时并提高理解要点命中数？SUS 评分如何？

RQ5（边界）：基于空闲时长、显式指令与提交动作的边界判定规则，在 6—8 个任务上的 F1 处于什么水平？阈值对结果有多敏感？

（二）研究目标

G1 提出任务中心八实体关联模型（1 个根实体 + 7 类记录实体：轮次、消息、工具调用、文件变更、测试运行、测试用例结果、检查点）与可扩展事件表示，给出与 PROV-O 的映射（RQ1）。G2 实现变更与消息的多对多关联，在 20—30 对样本上量化关联准确率（RQ2）。G3 实现 hunk 粒度的证据链与 1—2 跳双向查询（RQ3）。G4 实现边界判定与快照恢复（RQ4、RQ5）。G5 实现受证据绑定的摘要与进展整理并量化弃答率（RQ3）。G6 在 6—8 个任务、4—5 名受试者上量化系统对代码理解、原因分析、问题定位与任务衔接的作用（全部 RQ）。

（三）研究内容

为便于展开，以下将任务书的五项研究内容细分为六条：第（1）条对应第（1）项，第（2）（3）条对应第（2）项，第（4）条对应第（3）项，第（5）条对应第（4）项，第（6）条对应第（5）项。

（1）信息模型与统一事件表示：抽象开发任务、对话轮次、消息、工具调用、文件变更（hunk）、测试运行、测试用例结果与检查点八类实体及关系，设计可扩展事件模式与 SQLite 存储结构。

（2）上下文采集、归一化与任务边界判定：采集 Claude Code 的会话与工具调用、文件系统与 Git、测试报告；把编辑动作归一化为 hunk；研究基于空闲时长、显式指令、目标切换与提交动作的边界判定规则，并在 6—8 个任务上报告 F1。

（3）变更与对话的关联：研究基于时间邻近、显式引用（路径、符号、行号、命令、测试名）与文件级位置映射的多对多关联，处理行号漂移，输出置信度并

支持人工确认；在 20—30 对人工核对样本上报告关联准确率。

（4）变更溯源与证据链：以 hunk 为单位生成变更卡片，记录触发来源、实施过程、修改前后快照、验证依据与后续演化，支持 1—2 跳双向查询，并报告 1 万条事件规模下的查询响应时间。

（5）任务快照与任务续接：记录已完成（有通过测试支撑）、未完成（已提出未实施）与待验证（失败或未运行）三类事项，以摘要加清单呈现并可跳转到证据；界面限定为任务中心、任务时间线与变更证据侧栏三个视图，文件与代码片段视角作为时间线的过滤与详情呈现。

（6）大语言模型辅助与评估：生成任务摘要、整理开发进展，要求输出绑定证据标识、证据不足时弃答；评估选取缺陷修复、功能新增、代码重构三类共 6—8 个任务，以 4—5 名受试者的 SUS 评分与理解正确率作小样本主观评估（NASA-TLX 可选）。

（四）研究边界与不做事项

本课题不实现或训练代码智能体本身，不做通用 LLM 可观测性平台，不做云端多租户与团队协作，不做跨仓库聚合与多智能体调度。界面限定为 Web 形态的三个核心视图（任务中心、任务时间线与变更证据侧栏），文件与代码片段视角作为时间线的过滤与详情呈现；IDE 插件、第二来源适配列为可选，不计入必做范围。

（五）研究问题与研究内容的对应关系

各研究问题与目标、内容、关键问题、指标及论文章节的对应关系如表 2-1 所示。

表 2-1 研究问题—研究目标—研究内容—关键问题—评价指标—论文章节对应表

研究问题	研究目标	研究内容	关键问题	评价指标	论文章节
RQ1 表示	G1	（1）	K1	M1、M2	第 3 章
RQ2 关联	G2	（3）	K2	M3	第 4 章
RQ3 溯源	G3、G5	（4）（6）	K3	M5、M6、M10	第 5 章
RQ4 续接	G4、G5、G6	（5）（6）	K5	M7、M8、M9、M11	第 6 章
RQ5 边界	G4	（2）	K4	M4	第 4 章

（六）拟解决的关键问题

关键问题一：异构记录的统一表示与跨源链接（文本流、位置区间、结构化事件与层级结果集的时间粒度与标识体系不同）。

关键问题二：变更与对话的可靠关联（无显式声明，一次修改可能由多轮对话促成，行号漂移，并行任务交叉）。

关键问题三：证据链的可信呈现与幻觉抑制（结论须绑定真实证据，否则弃答，并报告弃答率）。

关键问题四：任务边界的可判定性与阈值敏感性。

关键问题五：恢复上下文的最小充分性（区分效果来自信息组织还是摘要生成）。

（七）创新点与特色

1. 面向智能体会话的任务上下文重建。与 Mylyn[23] 等以“显式激活任务 + IDE 代码元素”为前提的任务聚焦界面不同，本系统面向智能体事件流：边界未被显式声明时，依据空闲时长、目标切换、显式指令与提交动作判定边界，把跨会话的对话、变更、工具调用与测试挂接到同一任务。可验证差异：给定同一任务的两次会话记录，系统能自动归并并返回完整上下文集合。

2. 含验证依据的细粒度变更证据链。SZZ[18] 建立“变更—缺陷报告”单向关联，Codebook[26] 建立“代码—沟通记录”静态关联，二者都不含“由哪句指令触发、由哪次测试验证”。本系统以 hunk 为单位同时绑定触发消息、工具调用、测试运行与后续演化，可回答“为什么改、依据是什么、还有效吗”。

3. 面向中断恢复的恢复快照自动生成。已有研究给出恢复线索的类型与策略 [27-28]，但所需信息仍需人工搜集；本系统把恢复线索从“人工重建”转为“由任务上下文图自动生成”，以任务快照呈现已完成、未完成与待验证三类事项，并支持从摘要跳转到证据。

4. 受证据绑定约束的大模型辅助。要求每条摘要与进展结论绑定真实证据标识，证据不足时弃答，并把弃答率与不支持率并列报告，避免以少答换取低错误率。

（八）预期成果

1. 可运行的原型系统一套，含源代码、部署与使用说明，支持离线模式；

2. 毕业设计（论文）一份，约 1.5—2 万字；

3. 文献综述、外文资料翻译（按学院要求），系统设计文档、评估报告与复现脚本；

4. 6—8 个典型任务的实验数据与复现脚本；

5. 经脱敏的开发过程事件样例数据集（不少于 300 条事件、3 个完整会话），支撑可复现性与离线演示；

6. 典型任务的演示视频与答辩幻灯片。

2．研究方案

（一）研究方法

文献研究法：梳理代码智能体、软件仓库挖掘、过程溯源与人机交互四方向文献，明确空白与可复用方法。原型实现法：以可运行原型为载体迭代验证设计假设。实验研究法：在固定开源仓库上构造典型任务，通过对照实验量化效果。问卷与访谈法：用 SUS[42] 与半结构化访谈获取主观反馈，NASA-TLX[41] 作为可选的认知负荷观察。

（二）技术路线

研究按“需求与调研—模型设计—采集实现—关联与溯源—界面与续接—评估验证”六个步骤推进，前一阶段产出构成后一阶段输入，如图 2-1 所示：前两阶段产出需求清单、spike 报告与实体关系模型（后续工作的契约）；第三阶段打通“一次会话 → 文件变更 → 测试结果”的最小闭环；第四阶段实现关联算法与证据链查询；第五阶段转化为界面与任务续接；第六阶段检验效果并回流修正模型。六个步骤与表 2-5 的十一个阶段是两套编号：前者是研究推进的逻辑顺序，后者是各月任务与缓冲，不可混用。

（三）系统总体架构

系统采用四层架构，如图 2-2 所示：接入层从智能体事件接口、文件系统与 Git、测试报告抽取原始事件；模型与存储层把事件规范化后写入 SQLite 实体表与全文索引；分析与关联层完成归一化与边界判定、实体链接、证据链构建与大模型辅助分析；交互层提供任务中心、任务时间线与变更证据侧栏三个视图。大模型作为旁路能力被调用，输出经证据校验后才进入界面，校验失败则弃答。



图 2-1　研究技术路线

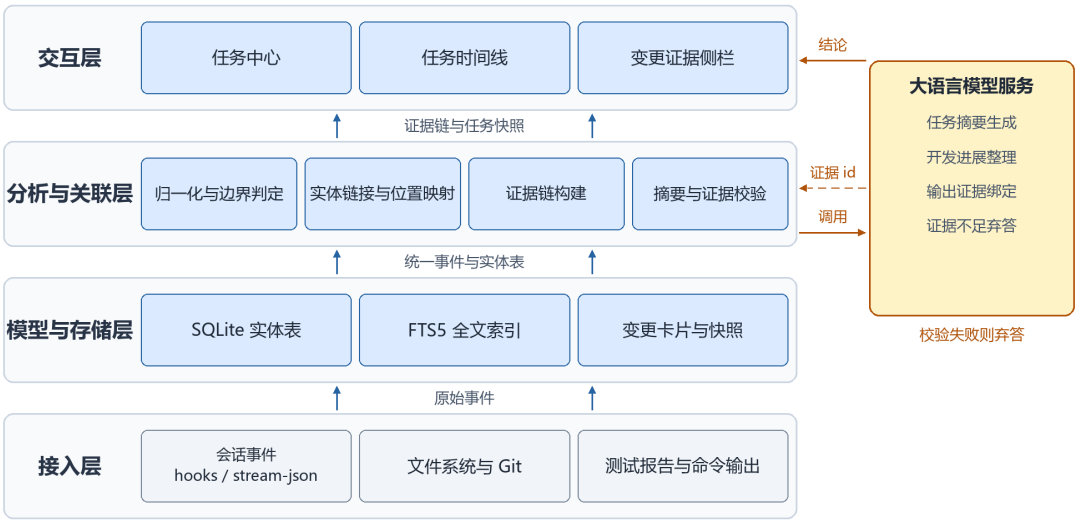


图 2-2　系统总体架构

（四）关键设计

信息模型以开发任务为根实体，挂接对话轮次与消息，横向关联由消息触发的工具调用、由工具调用产生的文件变更与由命令触发的测试运行，并以检查点记录任务快照。事件用 JSON 表示，字段参考 OpenTelemetry 生成式 AI 语义约定 [40] 并保留原始字段。表 2-2 给出各实体与 PROV-O[32] 的映射；这也回答了“为何不直接用 PROV”——PROV 提供通用抽象，但未规定任务边界如何判定、hunk 如何与消息多对多关联、测试结果如何作为验证依据挂接。

表 2-2 信息模型与 PROV-O 的映射

本课题概念	PROV-O 对应	说明
开发任务	prov:Activity	以目标为边界的一段过程
对话轮次、消息	prov:Entity	用户或智能体产生的文本实体
工具调用	prov:Activity	使用工具的活动，prov:used 指向参数
文件变更（hunk）	prov:Entity	由工具调用经 prov:wasGeneratedBy 生成
测试运行	prov:Activity	验证活动，prov:used 指向被验证的变更
测试用例结果	prov:Entity	验证活动的产出
智能体、开发者	prov:Agent	经 prov:wasAssociatedWith 关联活动
检查点	prov:Entity	任务阶段性快照

采集端的难点是编辑动作与 hunk 不同构：Claude Code 的 Edit 是“旧串 → 新串”，Codex 为补丁，Cursor 为整文件写入，Aider 为搜索替换块，均非 unified diff。为此设计六步归一化：取工具调用原始参数；在修改前文件中定位被替换片段（精确匹配失败则首尾锚定加模糊匹配）；应用替换得到修改后内容；对二者计算 unified diff 得到 hunk；与 Git 差异交叉校验，不一致时降低置信度；记录编辑格式元数据。

关联算法采用“规则打分—阈值判定—人工确认”三段式，规则含时间邻近、显式引用匹配、文件级位置映射与文本相似度。关联为多对多，区分 primary（工具调用直接产出该 hunk）与 secondary（消息提及该 hunk 涉及的文件或符号），变更卡片展开 primary、折叠 secondary 并显示计数。变更标识采用“内容哈希 + 文件路径 + 生成时间”三元组，避免行号漂移导致标识失效；文件级位置映射写入时计算、读取时按差异重放校正。

（五）实验与评估方案

（1）研究定位。本研究定位为小样本探索性 pilot：4—5 名受试者、6—8 个任务，样本不足以支撑确证性检验，故不设显著性结论、不做多重比较校正，全部推断以描述统计与逐人差值表述并标注“探索性”，把资源投入离线指标与错误分析。分析单位定为受试者而非单次观测。

（2）任务集与数据构造。实验在 2 个中小规模开源仓库（TypeScript 与 Python 各 1 个）上进行，共 6—8 个任务，覆盖缺陷修复、功能新增、代码重构三类（每类 2—3 个）；不少于三分之一改写自 SWE-bench Verified 与 SWE-rebench[43] 的真实 issue，其余为自构造，须预先给出预期变更文件与验证测试。任务由本人使用 Claude Code（官方钩子与 stream-json）执行，采集结果与 Git、测试报告合并落库；OpenHands 离线解析列为可选扩展，不计入必做。关联与边界由本人与一名同学独立核对 20—30 对样本，分歧协商；报告关联准确率，不做 κ。

（3）评价指标。指标口径、实验载体、数据来源与报告方式见表 2-3。表中不设“达标线”列：探索性定位下硬阈值不构成结论依据，也易掩盖区间。SUS 为必做的主观指标，NASA-TLX 作为可选的认知负荷观察，二者均作描述性报告。

表 2-3 评价指标定义与报告方式

编号	指标	定义与口径	载体与数据来源	报告方式
M1	事件类型覆盖率	模式覆盖的记录类型数 ÷ Claude Code 官方文档列出的类型数与 Git/测试记录类型总数（分母预先固定）	实验一 • metric_coverage.py	比例 + 区间
M2	文件级位置映射成功率	归一化后的编辑动作中，能与 Git 差异在同一文件同一行区间对齐的比例（以 Git 为参照）	实验一 • metric_mapping.py	比例 + 分层
M3	关联准确率	20—30 对人工核对样本中，系统 top-1 关联与人工判断一致的比例	实验一 • metric_link.py	比例 + 错误类型
M4	任务边界判定 F1	系统切分边界与人工标注边界在 ±1 条消息容差内的重合度（6—8 个任务）	实验一 • metric_boundary.py	F1 + 阈值敏感性

续表 2-3

编号	指标	定义与口径	载体与数据来源	报告方式
M5	溯源查询正确率	15 条预注册查询中，证据链同时含金标准触发消息与验证测试的查询占比（二元判定）	实验一 • metric_provenance.py	比例 + Wilson 区间
M6	不支持率与弃答率	50 条大模型结论中，证据不存在或语义不支持的比例，及同期弃答比例	实验一 • metric_llm.py	双指标并列
M7	任务恢复用时	4—5 名受试者阅读上下文并回答恢复性问题的用时（秒），B0/C2 逐人对比	实验二 • metric_user.py	逐人差值 + 中位数
M8	理解要点命中数	回答命中金标准要点的个数（0—5），要点由未参与实现者预先列出并冻结	实验二 • metric_user.py	逐人差值
M9	问题定位用时	从给定修改请求到受试者指出正确目标文件与函数的用时（秒）	实验二 • metric_user.py	逐人差值
M10	查询响应时间	1 万条事件下时间线首屏、1—2 跳证据链、中文检索的 P95（不含大模型）	实验一 • bench_query.py	分查询类型 P50/P95
M11	SUS 可用性评分	4—5 名受试者对系统的 SUS 评分（0—100）	实验二 • metric_user.py	逐人得分 + 中位数

（4）对比基线与实验设计。设两个条件：B0（基线）分别查看 Git 日志与差异、原始会话记录与测试报告；C2（完整系统，含证据链、跳转与任务快照）。B1（纯摘要基线）仅在离线对照中可选，不进入受试者实验，以控制单次到访时长。

受试者实验为 4—5 名受试者，每人完成 2 个任务（B0 与 C2 各 1 个），共 8—10 次观测；任务从缺陷修复、功能新增、代码重构三类中选取，条件与任务的分配在受试者间平衡。每人一次到访：任务各 30 分钟、条件间 10 分钟洗脱、说明与问卷 20 分钟，约 90 分钟；4—5 次到访共约 6—8 小时。因样本小，仅作描述性报告。

实验一为离线客观评测：20—30 对人工核对样本上评测关联准确率；15 条

预注册查询上评测溯源正确率，查询与金标准在实验前写入版本库并提交哈希；6—8 个任务上评测边界判定与阈值敏感性；1 万条事件库上评测查询性能；50 条结论上评测不支持率与弃答率。

实验二为受试者实验：4—5 名具备编程经验的同专业学生，在自中断情境下完成 2 个任务（B0/C2 各 1 个）并回答恢复性问题与 SUS 问卷；NASA-TLX 可选。采用自中断而非“第三方接手”，因后者与真实场景不等价，接手范式仅作补充观察。

关联评测协议：每个样本对由人工判断系统给出的关联是否正确；准确率 = 正确对数 ÷ 总对数；错误按六类编码统计。不做 P/R/F1 与 κ。

（5）数据分析方法。以受试者为分析单位：每人得 B0 与 C2 的差值（任务类型尽量匹配，受试者间平衡任务与条件）。报告逐人差值点图、中位数与范围；样本 4—5 人，仅作描述性报告，不作推断，不设显著性结论。分类指标报告比例与 Wilson 区间；退出不补并报告完成率；表 2-3 的数值均由 experiments/ 下脚本生成。

（6）错误分析与效度威胁。从实验一抽取 20 个失败案例，两名编码者按六类编码（时间邻近误配、行号漂移、多轮共同促成、并行任务交叉、工具调用无显式引用、模型编造）报告占比与典型案例。效度威胁有四：自构造任务可能引入实验者偏差，以三分之一外部 issue 与“构造—执行—标注—评测”分离缓解；单机单用户限制外推；受试者为同专业同学，存在从属关系，故由第三方招募、不涉及课程成绩、可无理由退出；n = 4—5 无法排除个体差异，结论限于方向性判断。

（7）研究伦理与数据合规。受试者签署知情同意书，含研究目的、流程与时长、采集范围、数据用途、可随时退出且不影响课程成绩。落库前脱敏：正则过滤常见密钥与令牌、密钥扫描工具扫描仓库与日志、人工抽样复核；受试者以匿名编号标识，映射表加密单存；实验仓库限公开开源项目。向第三方模型服务发送代码前做过滤并在界面明示，同时提供本地模型或规则版摘要作离线降级；受试者可单独选择不向第三方发送代码。录屏与日志保存至答辩后 6 个月。

（六）可行性分析

（1）技术可行性。所需技术均为成熟方案：React 与 Vite、TypeScript 与 Python、SQLite、unified diff 与 tree-sitter[44]；需实测的是数据采集与中文检索。中文检索不用 FTS5 默认分词器（它按空格切词，中文整段被视为一词），改用 trigram 分

词器（SQLite 3.34 起支持），两字词另留 LIKE 兜底，方案与性能在 Spike 3 验证。采集侧以 Claude Code 为主来源：它提供钩子与 stream-json，其本地 JSONL 被声明为内部格式、随版本变化，故只作降级通道 [36]；OpenHands 将每个事件序列化为独立 JSON 文件 [4]，列为可选扩展；Cline 以会话为单位存于本地 SQLite 并用影子 Git 记录快照 [38]；Cursor 命令行提供 NDJSON，IDE 无公开导出格式 [37]；Aider 仅提供 Markdown 与 Git 提交。据此采用“官方接口为主、日志直读为辅”策略，为主来源实现适配器并预留扩展点，以 fixtures 固化样例。运行环境为 Windows：路径大小写、长路径、换行符与 autocrlf 均影响位置映射，故统一.gitattributes、路径归一化与 CRLF 规范化，并在 M2 中按平台分层报告。

（2）数据可得性与可行性验证计划。三个 spike 在开题前完成（11.02—11.15，不计入执行期 58 人日），每项给出判者、测量方式、阈值与不达标处置。Spike 1（2 人日）采集闭环：用 Claude Code 在真实仓库完成一次“改一个文件并跑通一次测试”的最小任务，经钩子与 stream-json 落盘并与 Git 差异、测试 JSON 合并为统一事件表；本人执行、另一名同学按脚本复核，验收为 1 个完整会话、不少于 30 条事件、1 个 hunk、1 条测试运行，消息—工具调用—文件变更连通率达 100%，gitleaks 密钥扫描零命中；未达标则改为离线导入并下调评估规模。Spike 2（2 人日）差异对齐：20 个真实编辑动作归一化为 hunk 后与 Git 差异比对，覆盖片段多次出现、换行符差异、跨函数替换、被格式化工具二次修改；IoU 定义为“hunk 行区间与 Git 行区间的交集除以并集”，验收为平均 IoU ≥ 0.90、对齐成功率 $\geq 90\%$；未达标则退化为仅显式引用。Spike 3（1 人日）查询性能：8 核 16 GB 机器上构造 1 万条事件库，测时间线首屏、1—2 跳证据链与中文检索，冷、热缓存各测 3 次取 P95，验收为热缓存 P95 ≤ 1 秒；未达标则加索引或物化视图，仍不达标则降低“实时查询”的表述强度。

（3）前期基础。已完成选题调研与文献核验（44 篇，逐条核对 DOI、卷期与页码）；已搭建文档构建与图表脚本；已确认 Claude Code 的官方事件接口形态与保留期约束，并了解 OpenHands 的离线事件格式；已具备 React、TypeScript、Python 与 SQLite 开发经验。

（4）时间与工作量。课题周期 2026.11—2027.05：2026.11.02—11.15 完成开题，2026.12 为机动月（期末与其他事务，仅完成开发环境搭建、样例数据与文献补充，不计入执行期 58 人日），执行期为 2027.01.01—04.28，共 17 周；扣除春节 1 周低强度，按 16 个有效周计。按模块估算，必做约 58 人日（464 小时）：信息模型与存储 3、采集与归一化（1 来源）7、关联与边界判定 5、证据链与双

向查询 7、三视图界面 8、任务续接 2、大模型摘要与证据绑定 2、系统评估（6—8 任务、4—5 人）6、论文 14、集成/文档/中期/答辩 4。按 30 小时/周 × 16 周 = 480 小时，缓冲约 3%。另列可选 11 人日（第二来源离线适配 4、LLM 变更意图归纳 2、独立文件/片段视图 2、受试者扩到 6 人 2、NASA-TLX 1），不计入考核，时间富余再做。若每周只能投入 20—25 小时（折合 40—50 人日），则按“LLM 辅助 → 评估规模 → 界面视图 → 关联规则”的顺序降级，优先保证“采集—关联—证据链—任务续接”主闭环，并在论文中如实报告实际完成范围。

（5）风险与应对。主要风险、触发条件、应对措施及对结论的影响如表 2-4 所示。

表 2-4 主要风险、触发条件与应对措施

风险	触发条件	应对措施	对结论的影响
采集接口随版本变化或保留期到期	某来源连续两周无法稳定取到事件	官方接口为主并记录版本号；Spike 1 前置验证	降级为离线导入，RQ1 改述为离线可覆盖性，创新点 1 降为设计主张
关联准确率不达预期	M3 关联准确率低于 0.6	保留人工确认入口；报告置信度分布与错误类型	溯源降为候选推荐，RQ2 结论改为描述性
受试者招募不足	到访前一周不足 4 人	扩大到同院研究生；以离线指标为主	报告实际 n 与完成率，结论限定为个案观察
单次到访超时	实测超过 100 分钟	2 个任务减为 1 个；问卷移出到访	M7、M8、M9 观测数下降，只报描述统计
行号漂移导致位置映射失效	M2 低于 70%	以内容哈希为主键、行号辅助；读取时重放校正	位置映射关闭，证据链退化为仅显式引用
大模型幻觉或现场无网络	M6 不支持率高于 20%	证据 id 后置校验并弃答；预置回放与规则版摘要	界面标注“生成内容”，相关结论不作主张
Windows 路径与换行差异	IoU 或 M2 出现系统性偏差	统一.gitattributes 与路径归一化；CRLF 规范化	在效度威胁中报告，不影响主结论

3．工作进度安排

各阶段的工作内容与阶段成果见表 2-5，时间关系如图 2-3 所示。表中每个

执行阶段的时间跨度已包含缓冲；2026.11—12 为开题与机动期，2027.01.01—04.28 为执行期；三个 spike 安排在阶段一（11.02—11.15）内并行完成，不计入执行期 58 人日。

表 2-5 研究进度安排

阶段	时间	主要工作	阶段成果
一	2026.11.02—11.15	选题确认、文献调研与竞品分析；完成开题报告并参加开题答辩	文献综述初稿、开题报告、开题答辩
二	2026.11.16—12.31	机动（期末与其他事务）；完成开发环境搭建、样例数据与文献补充	开发环境、样例数据
三	2027.01.01—01.15	需求确认；信息模型与事件模式定稿；采集最小闭环	设计文档、采集 demo
四	2027.01.16—01.31	采集与归一化；任务边界判定；关联规则 v1	采集与关联原型
五	2027.02.01—02.14	证据链与变更卡片；双向查询；性能基线（含春节，低强度）	原型 v0.5
六	2027.02.15—02.28	任务中心、任务时间线、证据侧栏三视图	原型 v0.7
七	2027.03.01—03.15	任务续接；大模型摘要（可降级）；中期材料准备	原型 v0.9、中期报告初稿
八	2027.03.16—03.31	中期检查；6—8 个典型任务实验；4—5 人试点	中期检查通过、实验数据
九	2027.04.01—04.14	实验分析与图表；论文初稿（各章并行）	实验报告、论文初稿
十	2027.04.15—04.28	论文修改、查重、定稿；答辩材料	论文定稿、答辩材料
十一	2027.05 上旬	答辩（按学院安排）	答辩材料

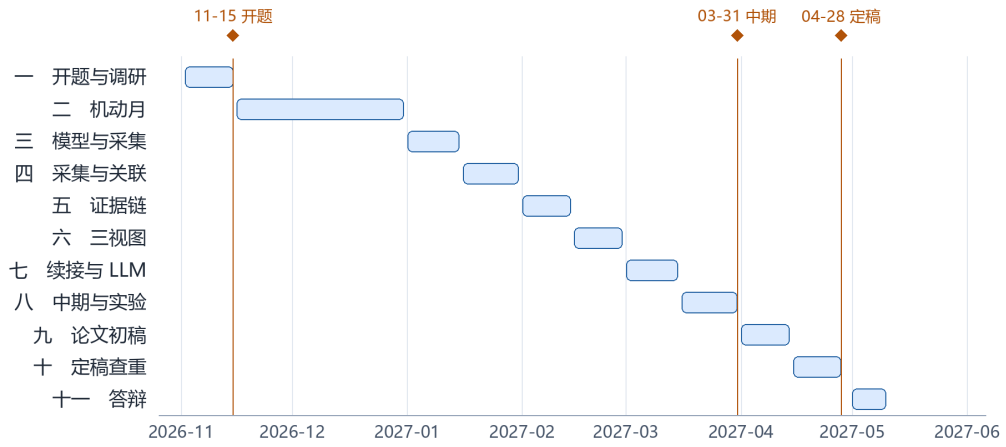


图 2-3 研究进度甘特图

三、毕业设计（论文）的主要参考文献

（说明：会议文献按 GB/T 7714-2015 著录，出版地与出版者从略；网络资源引用日期统一为 2026-10-27。）

[1] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: can language models resolve real-world GitHub issues?[C]//Proceedings of the 12th International Conference on Learning Representations. 2024. arXiv: 2310.06770.

[2] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code[EB/OL]. (2021-07-07)[2026-10-27]. https://arxiv.org/abs/2107.03374.

[3] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37. 2024. arXiv: 2405.15793.

[4] WANG X, LI B, SONG Y, et al. OpenHands: an open platform for AI software developers as generalist agents[C]//Proceedings of the 13th International Conference on Learning Representations. 2025. arXiv: 2407.16741.

[5] WANG X, CHEN Y, YUAN L, et al. Executable code actions elicit better LLM agents[C]//Proceedings of the 41st International Conference on Machine Learning. 2024. arXiv: 2402.01030.

[6] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: meta programming for a multi-agent collaborative framework[C]//Proceedings of the 12th International Conference on Learning Representations. 2024. arXiv: 2308.00352.

[7] QIAN C, LIU W, LIU H, et al. ChatDev: communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. 2024: 15174-15186. DOI: 10.18653/v1/2024.acl-long.810.

[8] XIA C S, DENG Y, DUNN S, et al. Agentless: demystifying LLM-based software engineering agents[EB/OL]. (2024-07-01)[2026-10-27]. https://arxiv.org/abs/2407.01489.

[9] HOU X, ZHAO Y, LIU Y, et al. Large language models for software engineering: a systematic literature review[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(8). DOI: 10.1145/3695988.

[10] FAN A, GOKKAYA B, HARMAN M, et al. Large language models for software engineering: survey and open problems[EB/OL]. (2023-10-05)[2026-10-27]. https://arxiv.org/abs/2310.03533.

[11] 张犬俊, 谢杨, 房春荣, 等. 检索增强生成在软件工程中的应用综述 [J]. 软件学报, 2026, 37(3): 1316-1339. DOI: 10.13328/j.cnki.jos.007567.

[12] 汪莹, 字千成, 彭鑫, 等. 基于大语言模型的故障复现测试用例生成方法 [J]. 软件学报, 2026, 37(4): 1690-1714. DOI: 10.13328/j.cnki.jos.007474.

[13] PENG S, KALLIAMVAKOU E, CIHON P, et al. The impact of AI on developer productivity: evidence from GitHub Copilot[EB/OL]. (2023-02-13)[2026-10-27]. https://arxiv.org/abs/2302.06590.

[14] MOZANNAR H, BANSAL G, FOURNEY A, et al. Reading between the lines: modeling user behavior and costs in AI-assisted programming[C]//Proceedings of the CHI Conference on Human Factors in Computing Systems. 2024: 1-16. DOI: 10.1145/3613904.3641936.

[15] PERRY N, SRIVASTAVA M, KUMAR D, et al. Do users write more insecure code with AI assistants?[C]//Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. 2023: 2785-2799. DOI: 10.1145/3576915.3623157.

[16] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: how programmers interact with code-generating models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI: 10.1145/3586030.

[17] VAITHILINGAM P, ZHANG T, SO S. Expectation vs. experience: evaluating the usability of code generation tools powered by large language models[C]//Proceedings of the CHI Conference on Human Factors in Computing Systems Extended Abstracts. 2022: 1-7. DOI: 10.1145/3491101.3519665.

[18] ŚLIWERSKI J, ZIMMERMANN T, ZELLER A. When do changes induce

fixes?[C]//Proceedings of the 2005 International Workshop on Mining Software Repositories. 2005: 1-5. DOI: 10.1145/1083142.1083147.

[19] KAGDI H, COLLARD M L, MALETIC J I. A survey and taxonomy of approaches for mining software repositories in the context of software evolution[J]. Journal of Software Maintenance and Evolution: Research and Practice, 2007, 19(2): 77-131. DOI: 10.1002/smr.344.

[20] ROBBES R, LANZA M. A change-based approach to software evolution[J]. Electronic Notes in Theoretical Computer Science, 2007, 166: 93-109. DOI: 10.1016/j.entcs.2006.06.015.

[21] FALLERI J R, MORANDAT F, BLANC X, et al. Fine-grained and accurate source code differencing[C]//Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering. 2014: 313-324. DOI: 10.1145/2642937.2642982.

[22] GRUND F, CHOWDHURY S A, BRADLEY N C, et al. CodeShovel: constructing method-level source code histories[C]//Proceedings of the 43rd International Conference on Software Engineering. 2021: 1510-1522. DOI: 10.1109/ICSE43902.2021.00135.

[23] KERSTEN M, MURPHY G C. Using task context to improve programmer productivity[C]//Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2006: 1-11. DOI: 10.1145/1181775.1181777.

[24] DRAGUNOV A N, DIETTERICH T G, JOHNSRUDE K, et al. TaskTracer: a desktop environment to support multi-tasking knowledge workers[C]//Proceedings of the 10th International Conference on Intelligent User Interfaces. 2005: 75-82. DOI: 10.1145/1040830.1040855.

[25] BAO L, YE D, XING Z, et al. ActivitySpace: a remembrance framework to support interapplication information needs[C]//Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering. 2015: 864-869. DOI: 10.1109/ASE.2015.90.

[26] BEGEL A, KHOO Y P, ZIMMERMANN T. Codebook: discovering and exploiting relationships in software repositories[C]//Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering. 2010: 125-134. DOI: 10.1145/1806799.1806821.

[27] PARNIN C, RUGABER S. Resumption strategies for interrupted programming tasks[J]. Software Quality Journal, 2011, 19(1): 5-34. DOI: 10.1007/s11219-010-9104-9.

[28] PARNIN C, DELINE R. Evaluating cues for resuming interrupted programming tasks[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. 2010: 93-102. DOI: 10.1145/1753326.1753342.

[29] SAFER I, MURPHY G C. Comparing episodic and semantic interfaces for task boundary identification[C]//Proceedings of the 2007 Conference of the Center for Advanced Studies on Collaborative Research. 2007: 229-243. DOI: 10.1145/1321211.1321235.

[30] ROBBES R, LANZA M. Characterizing and understanding development sessions[C]//Proceedings of the 15th IEEE International Conference on Program Comprehension. 2007: 155-166. DOI: 10.1109/ICPC.2007.12.

[31] CHENEY J, CHITICARIU L, TAN W C. Provenance in databases: why, how, and where[J]. Foundations and Trends in Databases, 2009, 1(4): 379-474. DOI: 10.1561/1900000006.

[32] W3C. PROV-O: the PROV ontology[EB/OL]. (2013-04-30)[2026-10-27]. https://www.w3.org/TR/prov-o/.

[33] SOUZA R, GUEROUDJI A, DEWITT S, et al. PROV-AGENT: unified provenance for tracking AI agent interactions in agentic workflows[C]//Proceedings of the 21st IEEE International Conference on e-Science. 2025. arXiv: 2508.02866.

[34] KO A J, MYERS B A. Designing the Whyline: a debugging interface for asking questions about program behavior[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. 2004: 151-158. DOI: 10.1145/985692.985712.

[35] PIMENTEL J F, MURTA L, BRAGANHOLO V, et al. noWorkflow: a tool for collecting, analyzing, and managing provenance from Python scripts[J]. Proceedings of the VLDB Endowment, 2017, 10(12): 1841-1844. DOI: 10.14778/3137765.3137789.

[36] Anthropic. Claude Code documentation: overview[EB/OL]. [2026-10-27]. https://code.claude.com/docs/en/overview.

[37] Cursor. Cursor documentation[EB/OL]. [2026-10-27]. https://cursor.com/docs.

[38] Cline. Cline: AI coding, open source and open choice[EB/OL]. [2026-10-27]. https://cline.bot.

[39] Langfuse. Observability overview[EB/OL]. [2026-10-27].

https://langfuse.com/docs/observability/overview.

[40] OpenTelemetry. Generative AI semantic conventions[EB/OL]. [2026-10-27]. https://opentelemetry.io/docs/specs/semconv/gen-ai/.

[41] HART S G, STAVELAND L E. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research[J]. Advances in Psychology, 1988, 52: 139-183. DOI: 10.1016/S0166-4115(08)62386-9.

[42] BROOKE J. SUS: a quick and dirty usability scale[M]//JORDAN P W, THOMAS B, WEERDMEESTER B A, et al. Usability Evaluation in Industry. London: Taylor & Francis, 1996: 189-194.

[43] BADERDINOV I, GOLUBEV A, NEKRASHEVICH M, et al. SWE-rebench: an automated pipeline for task collection and decontaminated evaluation of software engineering agents[EB/OL]. (2025-05-26)[2026-10-27]. https://arxiv.org/abs/2505.20411.

[44] Tree-sitter. Tree-sitter documentation[EB/OL]. [2026-10-27]. https://tree-sitter.github.io/tree-sitter/.

四、审核意见

指导教师意见：（课题难度是否适中、工作量是否饱满、进度安排是否合理、工作条件是否具备、是否同意开题等）

指导教师签名：_____ _____年____月____日

专业审核意见：（适当对各项内容给与意见和评价，字数不限。）

负责人签名：_____ _____年____月____日